PROBLEM STATEMENT

1. CAMPUS SCHEDULING AND VENUE MANAGEMENT

Chilika Institute of Technology (CIT) is a engineering college with about 1,000 students across three years and five branches. Its timetable is made in spreadsheets, and room bookings happen over calls and WhatsApp groups. This works until two people want the same room.

The worst week of the semester is Week 7. Regular classes are still running, midsems start the week after, and Spectra, the annual tech fest, needs the auditorium, the seminar halls, the open-air theatre and the computer labs. Most of the semester's clashes happen in this one week.

2. PROBLEM STATEMENT

Build a web platform for CIT that generates the semester timetable, manages venue bookings, and handles last-minute changes without creating new clashes.

The platform should be able to:

1. Generate a weekly timetable for every session in the dataset (lectures, tutorials and labs) with no clashes.
2. Schedule the midsem exams for the following week.
3. Accept venue requests written in plain English or Hinglish, turn them into a structured booking, and let the user check and edit it before submitting.
4. Approve, reject or suggest an alternative for each Spectra week booking request, following the priority rules in Section 4, with a reason for every decision.
5. Handle changes such as a faculty member going on leave, a room being closed or a new session being added. The platform should propose a fixed timetable that moves as few sessions as possible, show exactly what changed, and apply it only after an Admin approves. If there is no valid fix, it should say which constraints are in conflict.
6. Show timetables by student group, by faculty and by room, along with pending approvals, current conflicts and room utilisation.
7. Provide an AI assistant for faculty to request schedule changes using natural language prompts, outputting an alternative solution or explaining unavailability. All AI-generated schedule proposals must be strictly validated by underlying code before being applied.
8. Explain its decisions when asked. For example, why a class was put at 8 AM, or why a booking was rejected.
9. Let an Admin load the dataset and change files in the format given in the starter pack, and export the timetable, exams and booking decisions as schedule.json.
10. Support manual scheduling of events by authorized roles, ensuring that all manually scheduled events strictly follow the hard constraints.

3. CONSTRAINTS

The teaching week runs Monday to Friday, in nine one-hour periods: P1 starts at 08:00 and P9 at 17:00. Lunch is from 13:00 to 14:00, between P5 and P6. Periods P10 to P13 (18:00 to 22:00) are only for events.

HARD CONSTRAINTS :

- Every session is scheduled exactly once, with one day, one start period and one room.
- A faculty member can't be in two places at the same time.
- A student group can't be in two places at the same time. Combined lectures count for every group in them. Sessions in the same elective basket slot don't clash with each other, since each student attends only one of them.
- A room can't hold two things at the same time.
- Room capacity must be at least the session size. During exams, effective room capacity is halved for all capacity validation checks.
- The room type must match what the session needs. A physics lab only runs in the physics lab.
- Faculty unavailability must be respected.
- Room unavailability (maintenance etc.) must be respected.
- No session can run across lunch. Three-hour labs can only be in P2 to P4 or P6 to P8.
- Sessions that need an accessible room must get one.
- All sessions in the same elective basket slot run at the same day and period.
- A faculty member can't teach more than their daily limit (4 hours in the CIT data).
- An approved booking can't overlap with a session or another booking in the same room, or clash with a group it's mandatory for.
- A session with a fixed slot stays in that slot.

SOFT CONSTRAINTS:

- A faculty member is scheduled at a time they asked to avoid
- A student group has a free period between two classes on the same day (lunch doesn't count)
- A student group has more than 6 hours of class in a day
- A group has two lectures of the same course on the same day
- A session uses less than 40% of the room's capacity

- A session is scheduled in P9 (17:00)

4. ROLES AND PRIORITY

The platform should have four roles. Nobody can approve their own request.

Role	**What they can do**
Admin (Dean's office)	Everything. Generates the timetable, approves timetable changes and any booking that moves a class, can change the priority order.
Department coordinator	Requests changes to their department's sessions, books rooms for department events, approves bookings of their department's labs.
Faculty	Views their own timetable, marks unavailability, requests extra or moved sessions.
Club representative	Submits and cancels venue requests, and sees the status and reason for each.

When two requests want the same room at the same time, this order decides who gets it:

1. Exams
2. Regular classes
3. Institute events (Spectra main events, convocation)
4. Guest lectures
5. Department events
6. Club events

5. DATASET

All teams work on the same dataset, shared in the starter pack at the start of the hackathon. We have checked that a clash-free timetable exists for it.

What	**Count**
Rooms	27 (classrooms, lecture halls, auditorium, seminar halls, open-air theatre, 7 labs)
Faculty	40
Student groups	14 (Year 1 sections A to D, Year 2 and Year 3 for five branches)
Courses	62
Sessions to schedule	189 (243 teaching hours a week)
Spectra week booking requests	25 (one of them later cancelled)
Midsem exams	41

The starter pack contains:

- cit_public.json, the full dataset in one file: rooms (capacity, type, features, accessibility), room maintenance windows, faculty (unavailability, preferences, daily limit), student groups,

courses, all sessions to be scheduled, Spectra week booking requests with the original message text, cancelled requests, and midsem exams.

- The same data as CSV files, if you want to look at it in a spreadsheet.
- validator.py, the official checker. It only needs Python. It reports hard constraint violations, the soft penalty, the booking score, and how many sessions moved after a change.
- A README explaining every field and the export format, with a sample schedule.json.
- sample_disruption.json, a sample change file.
- Some sample booking requests in English and Hinglish.

Every session already has its faculty and student groups assigned. Your job is to give each one a day, a start period and a room.

6. IMPLEMENTATION RULES

- You can use any tech stack.
- You can use AI tools, both inside your product (LLM APIs, local models) and while building it (coding assistants, UI generators). Using them won't cost you marks. Not declaring them will.
- Declare all AI use in an AI_USAGE.md file in your repo. List every model or API your product calls and what it's used for, every AI tool you used to build the project and which parts it helped with, where your prompts are kept, and what the LLM is not allowed to decide in your system. If the judges find AI use that isn't declared, the team loses 10 marks.
- The LLM can read requests, ask follow-up questions, fill in forms and explain decisions. It can't be what decides whether a schedule or booking is valid. That has to be done by your own code, and every output from the LLM must go through those checks before it changes any data. If your validity checks depend on an LLM, you get 0 for scheduling correctness.
- Use your own API keys. If the LLM API fails or runs out of credits, the manual booking form must still work. Judges will check this.
- You can use open-source libraries and solvers (OR-Tools, Timefold etc.). List them in a THIRD_PARTY.md file. Projects you built earlier, including timetable projects from other hackathons, are not allowed.
- Create your GitHub repo after the hackathon starts. Commit history will be checked.

7. SUBMISSION REQUIREMENTS

- GitHub repo link. The repo must be public and contain:
- README.md with a short description, an architecture diagram, the tech stack, setup steps, the demo accounts, and known limitations
- .env.example with every environment variable (no real keys)
- AI_USAGE.md and THIRD_PARTY.md as described in Section 6
- an exports/ folder with schedule.json (generated by your app for the dataset) and schedule_after.json (after applying sample_disruption.json)

- Live deployed link. It must stay up until the results are announced. Judges will test your project on this link.
- Demo accounts for all four roles, listed in the README and working on the deployed link. Judges will log in with these. The Admin account should have a button to reset the demo data, so judges can start from a clean state.
- Video walkthrough of your working website, up to 5 minutes, as an unlisted YouTube link or a Drive link with view access. Show everything: generating the timetable, a plain-language request turning into a booking, a conflict being blocked, the Spectra week booking decisions, a rescheduling with its changes and approval, and the dashboard.

8. JUDGING CRITERIA

Criterion	**Marks**	**How it's judged**
Scheduling correctness	25	Validator run on your exports/schedule.json and on the timetable the judges generate in your app (timetable, bookings and exams). 25 marks minus 3 for every hard constraint violation, minimum 0.
Rescheduling	15	Judges apply their own change file in your app. 6 for no hard violations after the change, 5 for moving the fewest sessions compared to other teams, 4 for a clear view of the changes and a proper approval step.
Optimisation	10	Soft penalty from the validator, compared across teams that scored at least 15 in correctness.
Use of AI	15	Judges try the same set of tricky requests on every app: vague timings, conditions, priority abuse, Hinglish, attempts to get around the rules.
Booking decisions	10	Booking score from the validator, and whether each rejection comes with a reason that makes sense.
Usability and dashboard	10	Can a judge who has never seen your app find a room, read a timetable and approve a change, using only the README and the demo accounts?
UI and design	10	Look and feel of the frontend: consistent layout and styling, timetable grids that are easy to read, works on both laptop and phone screens, proper loading, empty and error states, and overall polish. Judged on the live app, not just the video.
Code and documentation	5	Setup works from the README, the code is reasonably organised, and the AI and third-party declarations are complete.

9. DISQUALIFICATION

A team will be disqualified if:

- The GitHub repo is private, deleted or can't be opened at any time between submission and the results.
- There is no live deployed link, or the link doesn't open a working app when the judges check it.
- The demo accounts are missing from the README or don't work, so the judges can't log in.
- No video is submitted, or the video doesn't show the working website.
- The repo was created before the hackathon started, or the project is built on code written before it, such as an old project, another hackathon's submission or a bought template.
- Code is copied from another team.
- Commits are pushed or the deployed app is changed after the submission deadline.
- The results shown in the video or in the exported files are made or edited by hand, hard-coded, or not actually produced by the app.
- Anyone outside the registered team, other than the official mentors, writes code or works on the project. Declared AI tools are fine.
- The team interferes with another team's work, accounts or deployment, or tampers with the validator or the organisers' files.